

• АРХИТЕКТУРНЫЙ СПРАВОЧНИК ДЛЯ ЗАЩИТЫ

PlagLens

Платформа автоматической проверки студенческих работ:
антиплагиат и LLM-анализ кода

Учебный проект · ФКН НИУ ВШЭ, ОП «Прикладной анализ данных»

Развёрнут в продакшене: <https://plaglens.ru>

Документ описывает **реализованную систему** (источник истины — код)

Версия от 5 июня 2026 г.

7

микросервисов
FastAPI

1

SPA-фронтенд
React 19

4

OAuth-провайдера
+ e-mail

Kafka

событийная
шина

Содержание

- 1 Что такое PlagLens
- 2 Архитектура в целом
- 3 Технологический стек
- 4 Сервисы по отдельности
- 5 Модель данных
- 6 Аутентификация, RBAC и мультитенантность
- 7 Антиплагиат: движок Dolos
- 8 LLM-анализ кода
- 9 Интеграции и импорт/экспорт
- 10 Событийная шина (Kafka)
- 11 Инфраструктура и развёртывание
- 12 Наблюдаемость и надёжность
- 13 Ключевые архитектурные решения и компромиссы
- 14 Ограничения и планы развития
- 15 Вероятные вопросы комиссии и ответы

01 Что такое PlagLens

PlagLens — многопользовательская (мультиотенантная) платформа для проверки студенческих работ по программированию: она принимает решения студентов, ищет заимствования между ними и анализирует код большой языковой моделью (LLM), помогая преподавателю быстрее и обоснованнее выставлять оценки — в одном месте, без ручной склейки между десятком вкладок.

Какую проблему решает

- Преподаватели CS-курсов вручную проверяют десятки/сотни решений, ищут списывание глазами и пишут однотипные комментарии — это долго и субъективно.
- Работы приходят из разных источников: Stepik, Яндекс.Контест, eJudge, прямые загрузки. Их нужно собрать в одном месте.
- Нужна не «чёрная коробка с процентом», а **объяснимый** результат: какие пары решений похожи, на чём именно, и что про код «думает» LLM.

Что система делает (сценарий преподавателя)

- Заводит курс и задания; импортирует условия и посылки из Stepik / Я.Контеста / eJudge или принимает прямые загрузки (код, ZIP, PDF).
- Запускает антиплагиат: попарное сравнение всех решений задания и сравнение со всем корпусом прошлых лет; получает кластеры подозрительных пар с подсветкой совпадений.
- Запускает LLM-анализ: краткое резюме решения, сигналы риска, вопросы на понимание — преподаватель сам решает, что показать студенту.
- Выставляет оценки и выгружает их в привязанную таблицу Google Sheets; студент видит свою оценку и комментарии в личном кабинете.

Роли

- **Студент** — отправляет решения, видит свои посылки, оценки и обратную связь.
- **Ассистент** — проверяет распределённые на него посылки, не управляет курсом.
- **Преподаватель** — владелец курса: задания, проверки, оценки, интеграции, экспорт.
- **Администратор** — управляет тенантом (организацией), пользователями, провайдерами входа и интеграциями.

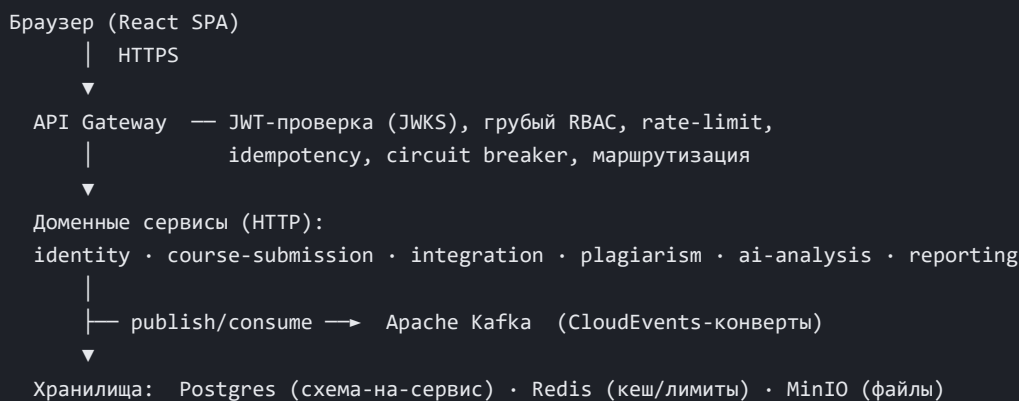
02 Архитектура в целом

PlagLens построен как набор из 7 независимых сервисов на FastAPI поверх общей доменной модели, с событийной шиной Apache Kafka, единым Postgres (схема-на-сервис), Redis для кеша и лимитов и MinIO для файлов. Весь стек поднимается одной командой `docker compose up`.

Стиль архитектуры

Микросервисы с **API-шлюзом** на входе и **событийной шиной** между сервисами. Синхронные операции (запросы пользователя) идут через шлюз по HTTP; асинхронные побочные эффекты (после загрузки посылки запустить антиплагиат, отправить уведомление, записать аудит) распространяются событиями через Kafka. Это даёт слабую связанность: сервис-источник не знает и не ждёт потребителей.

Поток запроса (сверху вниз)



Семь сервисов

Сервис	Зона ответственности
gateway	Единая точка входа: аутентификация, маршрутизация, лимиты, идемпотентность, агрегированный health.
identity	Пользователи, тенанты, RBAC, вход (e-mail + OAuth), 2FA, JWT-ключи, приглашения/коды.
course-submission	Курсы, задания/ДЗ, посылки, версии, оценивание, распределение между ассистентами.
integration	Адаптеры импорта (Stepik, Я.Контекст, eJudge, ручная загрузка), Google Sheets, Telegram-бот, расписания/вебхуки.
plagiarism	Оркестрация движка Dolos, попарные/кластерные результаты, кросс-курсовой корпус отпечатков, флаги подозрительности.
ai-analysis	LLM-анализ кода через OpenAI-совместимый API: кеш, бюджеты, failover, защита от prompt-injection.
reporting	Отчёты и экспорт оценок, дашборды-аналитика; вошёл в себя audit (журнал) и notification (e-mail/Telegram).

Почему именно так

Изначально доменов было больше (audit, notification и пара других — как отдельные сервисы). В ходе рефакторинга топология осознанно сведена к 7: course и submission слиты в course-submission (они всегда меняются вместе), а audit и notification стали внутренними модулями reporting (низкая нагрузка, общий жизненный цикл). Число сервисов — это компромисс между чёткими границами и эксплуатационной простотой, а не самоцель.

03 Технологический стек

Бэкенд (все сервисы)

Слой	Технология
Язык / рантайм	Python 3.12, асинхронный (asyncio)
Веб-фреймворк	FastAPI 0.110+, Uvicorn

Слой	Технология
Валидация / схемы	Pydantic v2, pydantic-settings
ORM / БД	SQLAlchemy 2.x (async) + asyncpg, миграции Alembic
Кеш / состояние	Redis (redis-py async)
События	Apache Kafka через aiokafka, конверты в стиле CloudEvents
Объектное хранилище	MinIO (S3-совместимое) — файлы решений, аватары
Аутентификация	PyJWT[crypto] RS256, argon2-cffi (пароли), authlib (OAuth), pyotp (2FA)
Наблюдаемость	structlog (JSON-логи), prometheus-client, OpenTelemetry/Jaeger
Общая библиотека	plaglens-common: ошибки, RBAC, сервис-токены, ServiceClient, секреты

Фронтенд (SPA)

Слой	Технология
Каркас	React 19 + TypeScript 5.6, сборка Vite 5
UI / стиль	Tailwind CSS v4 + shadcn/ui (Radix UI primitives)
Данные / запросы	TanStack Query 5 (кеш, инвалидация), axios
Маршрутизация	react-router-dom 6
Формы / валидация	react-hook-form + zod
Графики / таблицы	Recharts (дашборды), Univer (редактор Google-таблиц)
Прочее	katex (формулы в условиях), i18n RU/EN, lucide / react-icons
Тесты	Vitest (юнит), Playwright (e2e)

Инфраструктура

Слой	Технология
Оркестрация	Docker Compose (один сетевой контур, именованные тома)
Прокси / TLS	Traefik (dev), nginx + Let's Encrypt (прод), Cloudflare DNS
БД / кеш / файлы	PostgreSQL, Redis, MinIO
Шина	Kafka в режиме KRaft (без ZooKeeper)
Секреты	HashiCorp Vault (KV v2) с env-first-приоритетом
Метрики / трейсы	Prometheus + Grafana, Jaeger
CI/CD	GitHub Actions: ruff + мурп + pytest на сервис, сборка образов в GHCR

04 Сервисы по отдельности

Gateway — API-шлюз

FastAPI-реверс-прокси, единственная публичная точка входа. Не содержит доменной логики — отвечает за сквозные задачи:

- Аутентификация по JWT (проверка подписи по JWKS от identity) + список отозванных токенов в Redis.
- Грубый RBAC по глобальной роли (тонкий — внутри каждого сервиса).
- Маршрутизация на бэкэнд-сервисы по конфигу `gateway.yaml`.
- Rate-limit (token-bucket в Redis: по IP, по пользователю, по классу эндпоинта), CORS, лимит тела, request-id, трассировка.
- Кеш идемпотентности (повтор POST по Idempotency-Key), circuit breaker на каждый бэкэнд.
- Универсальный диспетчер `/v1/operations/{id}` для отслеживания асинхронных операций и агрегированный `/v1/health`.

Identity — пользователи и доступ

- Вход: e-mail + пароль (argon2) и OAuth (Google, Яндекс ID, GitHub, Telegram — OIDC; Stepik OAuth для импорта).
- JWT RS256: access 15 мин, refresh 30 дней; ключи публикуются через JWKS, которым шлюз проверяет подпись.
- RBAC: ровно 4 глобальные роли; курс-роли (owner/co_owner/assistant/student) едут в JWT.
- Мультитенантность: организации (тенанты) изолированы; есть тенант-плейсхолдер `public` для самозарегистрировавшихся.
- Приглашения и короткие коды: преподаватель генерирует код, студент активирует его и попадает в курс (с переносом из `public` при необходимости).
- 2FA по TOTP (pyotp), секрет шифруется Fernet; загрузка аватара в MinIO.

Course-Submission — курсы, задания, посылки

- Иерархия: курс → ДЗ (homework) → задание (assignment) → посылка (submission) с версиями.
- Приём решений: код, архивы, PDF; дедупликация по хешу содержимого; версии на одного автора.
- Оценивание: оценки, видимость для студента по флагу «release», распределение посылок между ассистентами (round-robin).
- Студенческие эндпоинты `/users/me/...` (свои посылки/оценки) и преподавательская очередь проверки.

Integration — импорт и внешние системы

- Адаптеры: Stepik, Яндекс.Контекст, eJudge, ручная загрузка (ZIP/CSV); общий интерфейс `import_problems / import_submissions`.
- OAuth-потоки к внешним системам, токены в Redis; импорт «как ДЗ» с фоновой задачей и прогрессом (Redis-операция, опрос модалькой).
- Google Sheets: привязка таблицы к курсу, выгрузка оценок (Service Account или OAuth преподавателя).
- Telegram-бот для уведомлений; расписания (cron) и входящие вебхуки; DLQ для разбора ошибок.

Plagiarism — антиплагиат

Оркестрирует движок **Dolos**, нормализует и хранит пары/кластеры, ведёт кросс-курсовой корпус фингерпринтов и жизненный цикл флага «подозрительно». Тяжёлые прогоны выполняются в очереди Celery. Подробно — раздел 7.

AI-Analysis — LLM-анализ

Анализирует код через OpenAI-совместимый API. Содержит защиту от prompt-injection, кеш по хешу кода, бюджеты на тенант/курс, авто-failover между провайдерами и «курирование» LLM-ответа в обратную связь студенту. Подробно — раздел 8.

Reporting — отчёты, аудит, уведомления

- Экспорт оценок (Google Sheets, CSV) с «умным» сопоставлением ФИО/логина и подсветкой ячеек перед записью.
- Дашборды-аналитика: активность, живые метрики (из Prometheus), распределение оценок.
- Встроенный **audit** (журнал событий по ресурсам/тенанту) и **notification** (e-mail через SMTP/Mailgun/Resend и Telegram), слушающие события из Kafka.

05 Модель данных

Один кластер PostgreSQL, но у каждого сервиса — своя схема (namespace). Это даёт логическую изоляцию доменов (сервис ходит только в свою схему) при простоте эксплуатации одного кластера. У каждого сервиса своя роль БД с доступом только к своей схеме.

Схемы (по сервисам)

Схема	Ключевые сущности
identity	users, tenants, oauth_identities, external_bindings, invitations, api_keys, totp
course	courses, homeworks, assignments, memberships, external_bindings
submission	submissions, submission_files, grades, feedback
integration	integration_configs, import_jobs, schedules, webhook_events, cursors
plagiarism	runs, pairs, clusters, corpus_fingerprints, suspicious_flags
ai_analysis	ai_analyses, budgets, providers, cache
reporting / audit / notification	exports, sheet_links, audit_events, email_transport_config, notifications

Принцип «изоляция данных»

Сервисы не лезут в чужие схемы напрямую — они общаются по HTTP (ServiceClient с сервис-JWT) или через события. Это сознательно: кросс-схемные SQL-запросы убирались в ходе рефакторинга в пользу HTTP, чтобы границы сервисов оставались настоящими, а не «общей базой под микросервисным соусом».

06 Аутентификация, RBAC и мультитенантность

Токены

- Вход выдаёт пару **access (RS256 JWT, 15 мин) + refresh (30 дней)**. Refresh хранит список отзыва в Redis.
- Identity — единственный, кто подписывает токены (приватный RSA-ключ). Все остальные **проверяют** подпись публичным ключом, который раздаётся через /.well-known/jwks.json.

- Шлюз проверяет токен на входе и кладёт в заголовки идентичность; сервисы доверяют этому и доуточняют права у себя.

Четыре глобальные роли

Роль	Что может
admin	Управление тенантом: пользователи, провайдеры входа, интеграции, аудит.
teacher	Полный контроль над своими курсами: задания, проверки, оценки, экспорт.
assistant	Проверка распределённых посылок; без управления курсом.
student	Свои посылки, оценки и обратная связь.

Помимо глобальной роли есть **курс-роли** (owner, co_owner, assistant, student) — права внутри конкретного курса. Они попадают в JWT, поэтому смена роли требует перелогина (флаг `requires_relogin`).

OAuth и вход

- Единый поток `authorization-code` + PKCE для всех провайдеров; стабильный `callback` `/api/v1/auth/oauth/{provider}/callback`.
- Telegram реализован как полноценный **OIDC** (`client_id/secret`, `id_token`), а не виджет — ради консистентности со всеми остальными провайдерами.
- Провайдер с пустыми кредитами считается «настроен, но не инициализирован»: сервис стартует, лишь этот вход отвечает 400.

Мультитенантность

Каждая организация — отдельный тенант; данные тенантов изолированы. Самозарегистрировавшийся пользователь попадает в тенант-плейсхолдер `public`; активация кода приглашения переносит его в тенант приглашающей организации. Сидовые тенанты: `public` и `system` (для платформенного админа).

07 Антиплагиат: движок Dolos

PlagLens намеренно не пишет свой детектор плагиата, а оркестрирует проверенный open-source движок Dolos (популярный, активно развивается). Это соответствует принципу «готовые библиотеки вместо велосипедов».

Как Dolos находит заимствования

- **Парсинг через tree-sitter**: код разбирается в абстрактное синтаксическое дерево (AST), а не сравнивается как текст. Поэтому переименование переменных, переформатирование и комментарии не сбивают детектор.
- **Токенизация + фингерпринты (winnowing)**: из AST строится поток токенов, по нему — устойчивые хеш-отпечатки (k-граммы). Сравниваются именно отпечатки.
- **Попарное сходство**: для каждой пары решений считается доля общих фингерпринтов → процент сходства; строятся кластеры «подозрительно похожих».

Что добавляет PlagLens поверх движка

- Оркестрация прогонов (queued → running → completed/failed), хранение нормализованных пар/кластеров, soft-delete прогонов.
- **Кросс-курсовой корпус**: фингерпринты прошлых лет, чтобы ловить копии не только внутри группы, но и из старых потоков.
- Жизненный цикл флага «подозрительно» на отправке и подсветка совпадающих фрагментов в паре (объяснимость для преподавателя).
- Тяжёлые прогоны — в очереди **Celery** (брокер Redis), чтобы не блокировать API.

Формулировка для комиссии

«Мы не изобретали алгоритм детекции: взяли Dolos — он строит AST через tree-sitter и сравнивает структурные фингерпринты, устойчивые к косметическим правкам. Наш вклад — оркестрация, кросс-курсовой корпус, объяснимая выдача (пары с подсветкой) и интеграция в проверку.»

08 LLM-анализ кода

ai-analysis обращается к LLM через OpenAI-совместимый API (по умолчанию OpenRouter / модель gpt-4o-mini). Один и тот же клиент работает с OpenAI, vLLM, llama.cpp, а через прокси — с YandexGPT/GigaChat.

Что важно для защиты

- **Защита от prompt-injection**: код студента оборачивается в теги <student_code>, в системном промпте есть явный запрет исполнять инструкции из кода, плюс пост-проверка ответа.
- **Кеш**: ключ sha256(model + версия_промпта + хеш_кода + язык). Повтор не дергает LLM, а отдаёт кеш и шлёт событие ai.analysis.cache_hit — экономит деньги.
- **Бюджеты**: счётчики токенов на тенант и курс в Postgres; до запроса pre-check (429 BUDGET_EXCEEDED), после — обновление. LLM не бесконечно дорогой.
- **Failover**: N подряд 429/5xx — переключение на следующего провайдера по приоритету.
- **Курирование**: LLM-ответ — это превью; преподаватель решает, что превратить в обратную связь студенту (HTTP-вызов в course-submission).

Что выдаёт LLM

- Краткое резюме решения (что делает код, какой подход).
- Сигналы риска (подозрительные места, потенциальные проблемы).
- Вопросы на понимание — чтобы преподаватель мог проверить авторство в беседе.

09 Интеграции и импорт/экспорт

Импорт работ

Источник	Что и как импортируется
Stepik	Дерево курса (уроки/шаги), импорт шагов «как ДЗ», подтягивание посылок по OAuth.
Яндекс.Контест	Контесты, задачи, участники, посылки; пулинг по расписанию (вебхуков у Я.Контеста нет).
eJudge	Контест и задачи по токenu.
Ручная загрузка	ZIP / CSV / прямой файл (в т.ч. PDF) от студента или преподавателя.

Экспорт оценок в Google Sheets

- Привязка конкретной таблицы к курсу; запись через Service Account (на уровне тенанта) или OAuth-токен преподавателя.
- «Умное» сопоставление строк: ФИО → логин каскадом; структурный анализ листа (где какая задача), подсветка ячеек перед записью.
- История выгрузок и предпросмотр изменений — чтобы преподаватель видел, что именно запишется.

Надёжность импорта

Импорт-как-ДЗ запускается фоновой задачей, прогресс пишется в Redis-операцию и опрашивается модалькой. Фоновые задачи держатся за сильную ссылку (`spawn_tracked`), чтобы сборщик мусора не оборвал длинный импорт; счётчик «получено посылок» тикает в реальном времени; состояние переживает перезагрузку страницы.

10 Событийная шина (Kafka)

Асинхронные побочные эффекты распространяются событиями. Сервис-источник публикует факт («посылка создана») и ничего не знает о потребителях — те сами решают, что делать (запустить антиплагиат, уведомить, записать аудит).

Принципы

- Kafka в режиме **KRaft** (без ZooKeeper); конверты в стиле **CloudEvents** (type, source, subject, tenant_id).
- Версионирование типов событий: `...created.v1` — схему можно эволюционировать, не ломая потребителей.
- **Идемпотентность** потребителей и **DLQ** (dead-letter) на «ядовитые» сообщения, чтобы один сбойный меседж не вешал поток.

Примеры событий

Событие	Кто публикует → кто реагирует
<code>submission.submission.created.v1</code>	course-submission → plagiarism (прогон), ai-analysis, reporting/notification
<code>integration.import.started/completed/failed.v1</code>	integration → reporting (история), фронт (прогресс)
<code>ai.analysis.completed / cache_hit.v1</code>	ai-analysis → reporting (метрики/история)
identity user/role события	identity → reporting/audit (журнал), notification

11 Инфраструктура и развёртывание

Локально и в проде

- Весь стек — `docker compose up`: 7 сервисов + Postgres, Redis, Kafka, MinIO, Prometheus, Grafana, Jaeger, Vault, прокси.

- Каждый контейнер на старте ждёт готовности Postgres/Redis/Kafka, накатывает миграции Alembic, опционально бутстрапит админа (identity) и поднимает Uvicorn — без ручных шагов.
- Прод: <https://plaglens.ru>, настоящий сертификат Let's Encrypt, DNS через Cloudflare; секреты — только в server-side .env (в git не попадают).

Секреты

Сервисы читают секреты через `plaglens_common.secrets` (Vault KV v2) с **env-first**-приоритетом: явная переменная окружения всегда побеждает, Vault спрашивается только если она не задана, а недоступный Vault или плейсхолдер `REPLACE_ME` мягко деградируют в `None` — контейнер всё равно стартует.

CI/CD

GitHub Actions: на каждый сервис — `ruff check + ruff format --check + mypy + pytest` с покрытием; плюс валидация `docker-compose`, `markdownlint` и сборка `libs`. Тер `v*` собирает по образу на сервис в GHCR.

12 Наблюдаемость и надёжность

Механизм	Реализация
Логи	structlog — структурированный JSON, request-id сквозной
Метрики	prometheus-client на каждом сервисе → Prometheus → дашборды Grafana (RPS, p95, ошибки, лаг Kafka)
Трассировка	OpenTelemetry → Jaeger (сквозной trace по сервисам)
Health	/healthz, /readyz на каждом; агрегированный /v1/health на шлюзе
Защита от перегрузки	rate-limit (token-bucket в Redis), circuit breaker на бэкэнды
Идемпотентность	кеш по Idempotency-Key на шлюзе (защита от двойного POST)
Устойчивость БД	рискованные кросс-схемные чтения — в SAVEPOINT, чтобы сбой не ронял всю транзакцию

13 Ключевые архитектурные решения и компромиссы

Комиссия любит вопрос «почему так, а не иначе». Ниже — сознательные решения и их обоснование.

Микросервисы, а не монолит

Чёткие границы доменов, независимый деплой и масштабирование тяжёлых частей (антиплагиат, LLM) отдельно от лёгких. Цена — сложнее эксплуатация; её гасим единым `docker compose`, общей библиотекой и схема-на-сервис в одном Postgres.

Один Postgres со схемами, а не БД-на-сервис

Логическая изоляция (схема + роль на сервис) без операционной боли нескольких кластеров — разумный масштаб для учебной платформы. Путь к разделению открыт: схемы уже разнесены.

Готовый Dolos вместо своего детектора

Детекция плагиата — сложная и хорошо решённая задача; писать свой алгоритм хуже проверенного. Наш вклад — оркестрация, корпус и объяснимость.

Событийная шина для побочных эффектов

Загрузка посылки не должна ждать антиплагиат/LLM/уведомления. Источник публикует событие и отвечает мгновенно; потребители работают асинхронно.

Кросс-схемный SQL → HTTP

Прямые запросы в чужую схему ломают границы сервисов; их заменили на HTTP-вызовы (ServiceClient + сервис-JWT). Где исторический кросс-схемный probe остался — он обёрнут в SAVEPOINT и мягко деградирует.

OIDC для всех провайдеров входа, включая Telegram

Единый поток authorization-code + PKCE для Google/Яндекс/GitHub/Telegram — одинаковая модель (client_id/secret), меньше частных случаев в коде.

14 Ограничения и планы развития

Что осознанно не доделано

- Vault и Kafka в проде — в dev-конфигурации (один брокер, dev-режим Vault); для боевой эксплуатации нужен кластер и sealed-Vault.
- Антиплагиат — один движок (Dolos); второй для кросс-проверки — в планах.
- Покрытие автотестами растёт, но неравномерно по сервисам.

Куда развивать (прикладное)

- **Антиплагиат для текстов** — рефераты и отчёты, а не только код.
- **Интерактивная защита решения** — LLM задаёт студенту вопросы по его коду.
- **Больше экспортов** — Moodle/LMS и открытый API помимо Google Sheets.
- **Зрелость прод-инфраструктуры** — секреты в Vault, сквозная трассировка, рост тестов.

15 Вероятные вопросы комиссии и ответы

Краткие, по делу формулировки — опора для устного ответа.

В: Почему микросервисы для учебного проекта — не оверинжиниринг?

О: Они дают чёткие доменные границы и возможность масштабировать тяжёлые части (антиплагиат, LLM) независимо. Эксплуатационную сложность гасим единым docker compose, общей библиотекой и одним Postgres со схемами. Топологию сознательно сократили с большего числа сервисов до 7, убрав избыточные.

В: Как именно работает поиск заимствований?

О: Движок Dolos парсит код в AST через tree-sitter, строит структурные фингерпринты (winnowing) и сравнивает их попарно. Поэтому переименования и переформатирование не обманывают детектор. Мы оркестрируем прогоны, храним пары/кластеры, ведём кросс-курсовой корпус и подсвечиваем совпадения.

В: Что мешает студенту обмануть антиплагиат, поменяв имена переменных?

О: Сравнение идёт по структуре (AST/фingerprint), а не по тексту — косметика вроде переименований и форматирования сходство почти не снижает. Полная переработка алгоритма — да, но это уже другое решение, что и видно эксперту.

В: Не опасно ли отдавать код студентов в LLM? Промпт-инъекции?

О: Код оборачивается в теги `<student_code>`, системный промпт запрещает исполнять инструкции из кода, есть пост-проверка ответа. Передаётся фрагмент в защищённой обёртке; политика конфиденциальности это описывает.

В: LLM — это же дорого. Как контролируете расход?

О: Три механизма: кеш по хешу кода (повтор не дёргает модель), бюджеты-счётчики на тенант/курс с отказом 429 при превышении, и батч-анализ с предпросмотром оценки токенов перед запуском.

В: Что если LLM-провайдер недоступен?

О: Авто-failover: после N подряд ошибок 429/5xx сервис переключается на следующего провайдера по приоритету. Клиент OpenAI-совместимый, так что провайдера можно сменить без изменения кода.

В: Как устроена авторизация и почему ей можно доверять между сервисами?

О: Identity подписывает JWT (RS256) приватным ключом; остальные проверяют подпись публичным ключом из JWKS. Шлюз проверяет токен на входе, сервисы доуточняют права. Межсервисные вызовы — по короткоживущему сервис-JWT.

В: Как изолированы данные разных организаций (тенантов)?

О: Каждая сущность привязана к `tenant_id`; запросы фильтруются по тенанту пользователя из токена. Плюс схема-на-сервис и роль БД с доступом только к своей схеме. Кросс-тенантные действия — отдельные, контролируемые сценарии.

В: Зачем Kafka, если можно вызвать сервис напрямую?

О: Чтобы пользовательский запрос не ждал тяжёлых побочных эффектов и чтобы источник не зависел от потребителей. «Посылка создана» — одно событие, на которое независимо реагируют антиплагиат, LLM, уведомления и аудит.

В: Что произойдёт, если упадёт один сервис?

О: Шлюз с circuit breaker вернёт понятную ошибку вместо зависания; асинхронные потребители Kafka переварят накопленные события после восстановления; «ядовитые» сообщения уходят в DLQ. Health-эндпоинты показывают состояние.

В: Чем подтверждается, что это рабочая система, а не прототип?

О: Развёрнута в проде на `plaglens.ru` с настоящим TLS; реальные входы через Google/Яндекс/GitHub/Telegram; импорт из Stepik и Я.Контеста; выгрузка в Google Sheets; метрики в Grafana; CI с линтером, типами и тестами.

В: Почему взяли Dolos, а не написали свой движок или JPlag/MOSS?

О: Принцип «проверенные библиотеки вместо велосипедов». Dolos — открытый, на tree-sitter, активно поддерживается и легко встраивается. Внешние сервисы (MOSS/Codequiry) убрали как лишнюю зависимость; ценность — в нашей обвязке.

В: Какой у проекта главный технический вызов был?

О: Согласовать асинхронность: длинные импорты и прогоны не должны блокировать UI и не должны теряться. Решение — фоновые задачи с устойчивыми ссылками, прогресс в Redis, события в Kafka и идемпотентные потребители.

Этот документ описывает реализованную систему и предназначен как опора для ответов на защите. При расхождении источником истины является код.